

Logs Labeling

Table of Contents(目錄)

- [Logs Labeling](#)
 - [Table of Contents\(目錄\)](#)
 - [Framework of Project \(專案框架\)](#)
 - [Project Introduction \(專案介紹\)](#)
 - [Stucture of whole project \(專案整體結構\)](#)
 - [Details of Each Step \(各步驟細節\)](#)
 - [Preprocessing](#)
 - [Weighting](#)
 - [Concept Extraction](#)
 - [Sequence Clustering](#)
 - [Auto Labeling](#)
 - [Quick Start\(快速開始\)](#)
 - [Environment Setup](#)
 - [Configuration](#)
 - [Running the Pipeline](#)
 - [Update & Changelog \(更新日誌\)](#)

Framework of Project (專案框架)

Project Introduction (專案介紹)

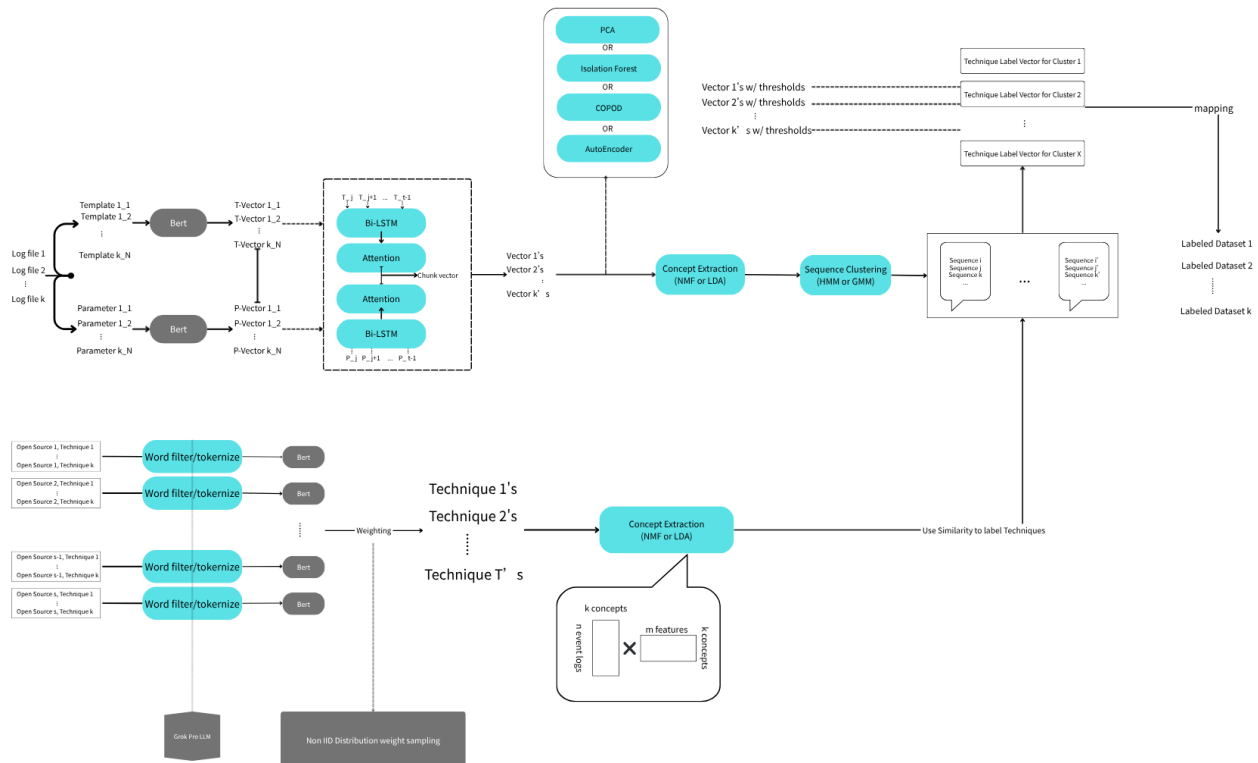
透過此專案，我們利用**機器學習**，對大量的日誌數據進行自動標註和分類，這將有助於提升日誌分析的效率，並為後續的數據挖掘和異常檢測提供有力支持。

目標是建立一個高效且準確的日誌標註系統，包含:

1. **數據預處理**: 透過 **Bert Embedding**對日誌數據進行向量化處理，將文本轉換為數值形式，以便於後續的機器學習模型訓練。
2. **序列區塊化** 利用 *時序性深度學習模型自動* 將日誌數據劃分為有意義的區塊，便於模型捕捉上下文信息。
3. **異常資料篩選**: 使用 **Isolation Forest** 方法識別並篩選出異常日誌數據做為後續標註的重點審查對象。

4. **概念提取**: 引用外來攻擊敘述文本作為比較基準，透過相似度計算比較日誌中的關鍵概念與對應攻擊手法的相似度作為標記依據。
5. **自動化標記**: 根據提取的概念和相似度計算結果，自動為日誌數據分配標籤，實現高效的日誌標註過程。
6. **評估與優化**: 對標註結果進行評估，並根據評估結果不斷優化標註算法和流程，以提升標註的準確性和效率。

Structure of whole project (專案整體結構)



- data/ : 儲存原始日誌數據和處理後的數據集
 - input_logs/ : 原始日誌數據
 - reference_resources/ : 參考資源，如MITRE ATT&CK文本
 - Intermediate_data/ : 各步驟產生的中間數據
 - processed_logs/ : 預處理後的日誌數據
- Data_Vizualization/ : 數據視覺化相關程式碼
- Text_Mining/ : 獲得外部參考文本的模組
 - Mitre_ATTCK/ : Mitre ATT&CK相關文本處理
 - LLM_Generation/ : 大型語言模型生成文本
 - Web_Scraping/ : 網頁爬取相關模組
- Logs_Labeling/ : 包含各個處理步驟的具體程式碼
 - preprocess/ : 數據預處理
 - sequence_blocking.py : 序列區塊化

- `anomaly_detection.py` : 異常資料篩選
- `concept_extraction.py` : 概念提取
- `auto_labeling.py` : 自動化標記
- `models/` : 儲存模型和相關資源
 - `bert_model/` : Bert Embedding 模型
 - `Unsupervised/` : 無監督學習模型
 - `HMM/` : 隱馬可夫模型相關
- `Visualization/` : 視覺化工具
- `config.py` : 專案參數配置
- `docs/` : 專案文件和說明

Details of Each Step (各步驟細節)

Preprocessing

詳見 [Preprocessing.md](#)

對於日誌數據的預處理步驟包括以下幾個主要部分:

1. **Templatize**: 將日誌序列分成模板與參數兩部分，模板部分保留文字資訊，參數部分以佔位符(E.g. `<*>`)表示。
2. **Vectorize**: 使用 Bert Embedding 將模板部分轉換為向量表示，便於後續的機器學習處理。
3. **Chunkize(optional)**: 利用Bi-LSTM+Attention模型，根據日誌的時序性將日誌序列劃分為多個區塊，以捕捉上下文信息。
4. **Concatenate**: 將模板向量與參數部分重新組合，形成最終的預處理日誌表示。
5. **Add anchoring**: 在預處理後的日誌表示中添加錨點，以便於後續的標註和分析。
6. **Save**: 將預處理後的日誌數據保存到指定位置，供後續步驟使用。

對於外部參考文本的預處理則包括:

1. **Text Cleaning**: 清理文本數據，去除無關字符和格式化問題。
2. **Tokenization**: 將文本分割成詞語或子詞單位，便於後續的語言模型處理。
3. **Embedding**: 使用 Bert 模型將文本轉換為向量表示。
4. **Save**: 將預處理後的參考文本保存，以供概念提取步驟使用。

Weighting

詳見 [Weighting.md](#)

對於日誌數據的加權步驟包括以下幾個主要目的:

- 透過全域的頻率資訊，提升關鍵日誌模板在相似度計算中的影響力。
- **核心想法:** 在不同來源的日誌都包含的序列，通常意味
 1. 並不是異常行為， 或者
 2. 並不是能夠區分攻擊行為的關鍵日誌。因此，這些日誌模板在相似度計算中應該被降低權重。
- 方法: 透過非監督式學習的方式，找出這些全域頻率較高的日誌模板，並根據結果作為加權的依據。

對於外部參考文本的加權目的:

- 不同來源的參考文本，其重要性可能有所不同。例如，某些攻擊手法的描述可能更為詳細和準確，這些文本在概念提取中的影響力應該更大。
- 本專案目的在將序列資料標記為 *Mitre ATT&CK* 的 Technique，因此 *Mitre ATT&CK* 的文本應該被賦予較高的權重。
- 另外，其他來源資料分布為 non i.i.d.，因此需要透過加權的方式，調整不同來源文本在相似度計算中的影響力，希望能藉此找出不同文本的全域概念。

Concept Extraction

詳見 [Concept_Extraction.md](#)

1. 透過NMF(Non-negative Matrix Factorization)方法，從預處理後的日誌數據中提取潛在的概念。 $(X = H * W)$
 - X: 預處理後的日誌數據矩陣 (m x n)
 - H: 日誌與概念的關聯矩陣 (m x k)
 - W: 概念與特徵的關聯矩陣 (k x n)
- 其中k為假定一個數據隱含概念的數量，可以根據實際需求進行調整。

Sequence Clustering

詳見 [Sequence_Clustering.md](#)

- 將數據序列將假定為由多個隱藏狀態所組成，並利用**隱馬可夫模型(Hidden Markov Model, HMM)**進行建模與訓練。
- 透過HMM模型，捕捉數據序列中的時序特性，並將相似的序列歸類到同一群組中。

Auto Labeling

詳見 [Auto_Labeling.md](#)

- 透過比較日誌數據中的概念與外部參考文本中的概念之間的相似度，進行自動標註。

Quick Start(快速開始)

Environment Setup

Configuration

Running the Pipeline

Update & Changelog (更新日誌)

- 2025-11-16: Init
- 2025-11-21:
 - 創立專案架構: [Readme.md](#)
 - 新建preprocess與Drain作為預設parser for Logs
- 2025-11-25:
 - 更新 [Readme.md](#)，補充外部文件爬蟲相關架構
 -